CIENCIA DE DATOS PARA ECONOMÍA Y NEGOCIOS · FCE-UBA

Organización de proyectos, APIs y fechas

Buenas prácticas para proyectos de datos — en R y en Python

Basada en la clase de Nicolás Sidicaro · Versión actualizada: Maida Beltran · Octubre 2026

EL PROBLEMA

Lo más típico que nos pasa al principio

```
analisis.R
analisis_v2.R
analisis_v2_final_FINAL2.R
modelo_final_ahora_si.ipynb
# ¿cuál es el bueno?
```

- No sé qué script hay que correr primero y cuál después.
- Cambié algo y ahora nada funciona (y no sé qué cambié).
- Un colega me pide el código y no le corre en su computadora.
- Tres meses después, no entiendo mi propio código.

La buena noticia: se resuelve con hábitos simples. No es difícil; solo hay que conocerlos.

LA IDEA DE TODA LA CLASE

Un proyecto está bien organizado si otra persona —o vos dentro de tres meses— puede correrlo y obtener lo mismo sin preguntarte nada.

R Python En toda la clase: cada práctica en los dos lenguajes. Cambian las herramientas; los principios, no.

HOJA DE RUTA

Qué vamos a ver

01 Estructura de carpetas

02 Moldes de proyecto: Cookiecutter y sus equivalentes en R

03 Scripts encadenados y reproducibilidad

04 Buenas prácticas de código

05 Entornos, dependencias y claves

06 APIs: BCRA y Banco Mundial

07 Fechas: lo esencial

01

Estructura de carpetas

Cómo organizar un proyecto de datos, en cualquier lenguaje.

ESTRUCTURA DE CARPETAS

Lo que otra persona necesita saber para correr tu análisis

¿Dónde están los datos crudos?

Los originales, tal como llegaron.

¿Dónde están los scripts?

El código que transforma y analiza.

¿Dónde van los resultados?

Tablas y gráficos que salen del código.

¿En qué orden se corre todo?

Qué va primero y qué después.

Clave: los datos crudos son sagrados. Nunca se modifican: todo cambio se hace en un script que lee el crudo y genera *otro* archivo. Así, un error en la limpieza nunca destruye el original.

ESTRUCTURA DE CARPETAS

Una estructura recomendada

```
mi_proyecto/
  datos/crudos/              Originales: NO se tocan
  datos/procesados/          Generados por scripts
  scripts/01_descarga        Descarga o copia datos
  scripts/02_limpieza        Limpieza y transformación
  scripts/03_analisis        Modelos, tests, cálculos
  scripts/04_graficos        Visualizaciones
  resultados/tablas/         Tablas exportadas
  resultados/figuras/        Gráficos exportados
  utils/                     Funciones propias
  docs/                      Informes, presentaciones
  README.md                  Qué es y cómo se corre
  .gitignore                 Qué no se sube a git
```

Lo que cambia según el lenguaje

	R	Python
Scripts	.R	.py
Funciones	utils/funciones.R	utils.py
Marca la raíz	mi_proyecto.Rproj	pyproject.toml
Notebooks	.qmd / .Rmd	.ipynb

Todo lo demás es idéntico: la estructura es del proyecto, no del lenguaje.

ESTRUCTURA DE CARPETAS

El README: lo primero que alguien lee

Contenido mínimo

- **Título y descripción:** una oración sobre qué hace.
- **Datos:** de dónde salen y cómo obtenerlos.
- **Instrucciones:** qué correr y en qué orden.
- **Dependencias:** qué paquetes hacen falta.
- **Autor/a y fecha.**

```
# Desempleo por provincia (EPH)
Calcula la tasa de desempleo por provincia.
## Datos
EPH (INDEC). Los baja scripts/01_descarga.
## Cómo correrlo
1. Abrir el proyecto (.Rproj o la carpeta)
2. Correr run_all.R (o python run_all.py)
## Dependencias
tidyverse, lubridate (ver renv.lock)
Autora: [nombre] · 2026-10-13
```

Para el TP: el README tiene que permitir que el docente reproduzca todo el trabajo.

ESTRUCTURA DE CARPETAS

Rutas absolutas: el error más común

La ruta absoluta funciona en una sola computadora. La relativa funciona en cualquiera.

R

```r
# MAL: solo anda en mi computadora
read.csv("C:/Users/nico/Desktop/eph.csv")
# BIEN: relativa a la carpeta del proyecto
read.csv("datos/crudos/eph.csv")
```

Python

```python
# MAL: solo anda en mi computadora
pd.read_csv("C:/Users/nico/Desktop/eph.csv")
# BIEN: relativa a la carpeta del proyecto
pd.read_csv("datos/crudos/eph.csv")
```

Usá siempre / en las rutas, aunque estés en Windows: R y Python la entienden. La barra invertida \ trae problemas en los dos.

ESTRUCTURA DE CARPETAS

¿Dónde está la carpeta del proyecto?

Opción	R	Python	Comentario
Abrir el proyecto	doble clic en mi_proyecto.Rproj	abrir la carpeta en VS Code, Positron o Jupyter	La más simple
Fijarla a mano	setwd("C:/.../cdd")	os.chdir("C:/.../cdd")	Funciona, pero si queda escrita en el script, lo ata a tu compu
Que la busque sola	here::here("datos", "eph.csv") — o fijar la ruta del proyecto en la consola con setwd() y luego usar rutas relativas	Path(__file__).resolve().parents[1] / "datos"	La más robusta; en Python, sube de scripts/ a la raíz

Crear un .Rproj

En RStudio: File › New Project › Existing Directory, y elegir la carpeta.

Ojo en Python

Path(__file__) solo existe en scripts .py. En un notebook, la carpeta de trabajo es la del notebook.

02

Moldes de proyecto

Cookiecutter y sus equivalentes en R: las buenas prácticas, escritas en una plantilla.

MOLDES DE PROYECTO

¿Qué es Cookiecutter?

1. El molde

Una carpeta modelo con espacios en blanco. La arma una vez alguien que sabe cómo organizar un proyecto.

2. Tus respuestas

Cookiecutter te pregunta el nombre del proyecto, el autor y qué herramientas querés usar.

3. Tu proyecto

Una carpeta lista para trabajar, con la estructura y los archivos básicos ya armados.

Como un cortante de galletitas: el molde es siempre el mismo, y cada galletita sale con la misma forma. **Cookiecutter es una herramienta de Python, pero el molde puede tener archivos de cualquier lenguaje:** R, Python, SQL, Markdown.

Para qué sirve: que todos los proyectos de un equipo —o todos los TP de una materia— arranquen con la misma estructura y las mismas buenas prácticas.

MOLDES DE PROYECTO

Cómo funciona: preguntas, respuestas, proyecto

El molde pregunta	Vos respondés	Dónde aparece en tu proyecto
nombre_proyecto	desempleo_eph	El nombre de la carpeta y el título del README
autor	Grupo 3	«Autor/a: Grupo 3» en el README
descripcion	Desempleo por provincia.	La primera línea del README

Adentro del molde, cada respuesta tiene su lugar marcado con llaves dobles: {{ cookiecutter.autor }}. Cookiecutter reemplaza cada una por lo que respondiste, en nombres de carpetas, de archivos y en el texto.

Las preguntas y sus valores por defecto viven en un archivo del molde llamado cookiecutter.json.

COOKIECUTTER DATA SCIENCE

Las decisiones que te hace tomar el molde

Pregunta	Qué decide	Opciones
project_name · author_name	Nombre del proyecto y autor	texto libre
environment_manager	Cómo se aísla el entorno	virtualenv, conda, uv, poetry…
dependency_file	Dónde se declaran los paquetes	requirements.txt, pyproject.toml…
testing_framework	Con qué se escriben los tests	pytest, unittest o ninguno
linting_and_formatting	Qué herramienta arregla el estilo	ruff o flake8 + black + isort
dataset_storage	Dónde viven los datos pesados	ninguno (local), S3, Azure o GCS
open_source_license · docs	Licencia y documentación	MIT, BSD-3 · mkdocs

Cada pregunta es una buena práctica: decidir desde el principio cómo aislar el entorno, dónde declarar los paquetes y si va a haber tests.

COOKIECUTTER DATA SCIENCE · GENERADA CON CCDS 2.3.0

La carpeta que arma

`data/raw/`	Lo original. No se toca.
`data/external/`	Datos de terceros.
`data/interim/`	Pasos intermedios.
`data/processed/`	Listo para modelar.
`notebooks/`	Explorar. Nombre: 1.0-iniciales-descripcion
`models/`	Modelos entrenados y predicciones.
`reports/figures/`	Gráficos para informes.
`references/`	Diccionarios de datos, manuales.
`project_name/`	El código: config, dataset, features, modeling, plots.
`docs/`	Documentación (mkdocs).
`README.md · pyproject.toml`	Qué es y qué necesita para correr.
`Makefile`	Atajos: make requirements, make lint, make format.
`.gitignore · .env`	Ya ignora /data/; .env para las claves.

COOKIECUTTER DATA SCIENCE

Las opiniones que trae el molde

Una receta en un solo sentido

Cada paso usa el anterior y nunca vuelve atrás: se puede rehacer todo desde el principio.

Lo crudo no se toca

Nunca se edita, y menos en Excel. Se lee y se transforma en otro archivo.

Los datos no van a git

data/ está ignorado por defecto; el README explica cómo conseguirlos.

Notebooks para explorar

Lo que se repite pasa a código fuente, que se puede testear y revisar.

Primero, el entorno

Mismas librerías y mismas versiones; si no, no es el mismo análisis.

Secretos fuera del código

Claves y contraseñas van en un archivo .env que nunca se sube.

Y una séptima: **si algo del molde no te sirve, borralo.** Se adapta el molde al proyecto, no al revés.

MOLDES EN R

Plantillas y moldes que se pueden usar en R

Herramienta	Qué arma	Cómo se arranca	Ideal para
cookiecutter (R)	Usa moldes cookiecutter desde R	cookiecutter::bake(molde, destino)	Un molde propio de la materia
usethis	Un .Rproj y una carpeta R/	usethis::create_project("mi_tp")	Arrancar simple: el TP
ProjectTemplate	Estructura completa y carga automática	library(ProjectTemplate); create.project("mi_tp")	Análisis grandes y repetitivos
targets	El pipeline como receta, con su grafo	targets::use_targets()	Cuando correr todo tarda
rrtools	Un paper con forma de paquete de R	rrtools::use_compendium("paper")	Papers académicos
Quarto	Informes, libros o sitios web	quarto create project	Informes reproducibles
golem	Apps Shiny con forma de paquete	golem::create_golem("app")	Tableros en producción

Para el TP alcanza con usethis::create_project() o con abrir un .Rproj. El resto, cuando el proyecto lo pida.

MOLDES EN R

Un molde propio de la materia, usado desde R

R · paquete cookiecutter

```
library(cookiecutter)
bake("molde_tp", "mis_proyectos",
  extra_context = list(
    nombre_proyecto = "desempleo_eph",
    autor = "Grupo 3",
    descripcion = "Desempleo por provincia."),
  no_input = TRUE)
```

Python · cookiecutter

```
$ pip install cookiecutter
$ cookiecutter molde_tp
# pregunta nombre, autor y descripción
```

Ojo con el paquete de R

No copia archivos ocultos (.gitignore, .gitkeep) ni carpetas vacías.
Solución: un LEEME.md en cada carpeta.

Resultado: una carpeta desempleo_eph/ con el README ya completo, run_all.R, scripts/, datos/crudos/, datos/procesados/ y resultados/.

El mismo molde funciona con R y con Python.

MOLDES DE PROYECTO

Otra opción: pedírselo a un agente de IA

Claude Code, Codex, Cursor o Antigravity pueden armar la estructura a partir de un pedido escrito.

Un pedido de ejemplo

```
Creá la estructura de carpetas para un proyecto de análisis en R sobre desempleo con la EPH:
datos/crudos, datos/procesados, scripts numerados del 01 al 04, resultados/tablas y
resultados/figuras. Agregá un README con las secciones Datos, Cómo correrlo y Dependencias,
un .gitignore que excluya datos/ y .Renviron, y un run_all.R que corra los scripts en orden.
No inventes datos ni análisis: solo la estructura.
```

A favor

Es rápido y se adapta a lo que pidas: otra estructura, otro lenguaje, otro README.

Cuidado

Revisá siempre lo que creó. Y guardá el pedido en el README: así se puede repetir y funciona como tu molde.

03

Scripts encadenados y reproducibilidad

Partir el trabajo en etapas y poder correrlo todo de nuevo.

REPRODUCIBILIDAD

¿Qué es un análisis reproducible?

Otra persona toma tus datos y tu código, lo corre, y obtiene **exactamente los mismos resultados**.

Credibilidad

Un informe económico que no se puede replicar no es creíble.

Actualización

Si llega un nuevo trimestre de la EPH, un flujo reproducible se actualiza en minutos.

Menos dolores de cabeza

Volver a correr todo es apretar un botón, no reconstruir pasos de memoria.

Referencia: Wilson et al. (2017), «Good Enough Practices in Scientific Computing», PLoS Computational Biology 13(6).

SCRIPTS ENCADENADOS

Un script gigante vs. varios scripts por etapa

Un script de 500 líneas

Difícil de navegar, de encontrar errores y de mantener. Si falla la línea 430, hay que correr todo de nuevo.

Etapas separadas

Cada etapa en su archivo. Si cambia la limpieza, se corre desde la limpieza en adelante.

Flujo típico (vale para .R o .py):

`01_descarga` baja o copia datos → `02_limpieza` limpia y transforma → `03_analisis` modelos y cálculos → `04_graficos` visualizaciones

El puente entre etapas son los archivos intermedios en datos/procesados/: cada script lee lo que dejó el anterior.

SCRIPTS ENCADENADOS

Cada script guarda; el siguiente levanta

`02_limpieza`
lee el crudo

→

`02_ocupados.csv`
guardado en datos/procesados/

→

`03_analisis`
lo levanta del disco

La regla: un script nunca usa objetos que creó otro script en la memoria. Si 03 necesita algo de 02, 02 lo guarda en un archivo y 03 lo lee de ese archivo.

Por qué

Cada script se puede correr solo: para revisar el análisis no hace falta repetir la descarga ni la limpieza.

Cómo nombrar lo que se guarda

Con el número del script que lo generó: 02_ocupados.csv sale de 02_limpieza.

SCRIPTS ENCADENADOS

Rutas de entrada y salida: instub y outstub

R

```r
# 02_limpieza.R
instub <- file.path("datos", "crudos")
outstub <- file.path("datos", "procesados")
archivo <- "eph_2025t3.csv"
eph <- read_csv(file.path(instub, archivo))
ocupados <- eph |> filter(ESTADO == 1)
write_csv(ocupados,
  file.path(outstub, "02_ocupados.csv"))
# 03_analisis.R
instub <- file.path("datos", "procesados")
ocupados <- read_csv(
  file.path(instub, "02_ocupados.csv"))
```

Python (from pathlib import Path)

```python
# 02_limpieza.py
instub = Path("datos") / "crudos"
outstub = Path("datos") / "procesados"
archivo = "eph_2025t3.csv"
eph = pd.read_csv(instub / archivo)
ocupados = eph[eph["ESTADO"] == 1]
ocupados.to_csv(outstub / "02_ocupados.csv",
  index=False)
# 03_analisis.py
instub = Path("datos") / "procesados"
ocupados = pd.read_csv(instub / "02_ocupados.csv")
```

Instub: de dónde lee el script.
Outstub: dónde guarda.
Ejemplo completo en el repositorio del curso (link en el README de la materia).

SCRIPTS ENCADENADOS

Un script maestro que corre todo

R

```r
# run_all.R — corre el proyecto completo
source("scripts/01_descarga.R")
source("scripts/02_limpieza.R")
source("scripts/03_analisis.R")
source("scripts/04_graficos.R")
```

Python

```python
# run_all.py — corre el proyecto completo
import subprocess
import sys
scripts = ["scripts/01_descarga.py",
  "scripts/02_limpieza.py",
  "scripts/03_analisis.py",
  "scripts/04_graficos.py"]
for script in scripts:
  subprocess.run([sys.executable, script],
    check=True)
```

Condición: cada script es autocontenido. Carga sus paquetes, lee sus archivos y no depende de objetos que quedaron en memoria.

04

Buenas prácticas de código

Nombres, estilo y comentarios.

BUENAS PRÁCTICAS DE CÓDIGO

Nombres de objetos

Regla	R	Python
snake_case	ingreso_horario, tasa_empleo	ingreso_horario, tasa_empleo
Nombres descriptivos	df_limpia dice más que d2	df_limpia dice más que d2
No usar nombres de funciones	data, mean, filter, c, T	list, sum, id, type, str
Guía oficial	Tidyverse Style Guide	PEP 8

En R

mean <- 5 no rompe mean(x): R busca la función por separado. Pero T <- 0 sí rompe T como atajo de TRUE.

En Python

sum = 10 y después sum([1, 2]) da TypeError: el nombre quedó pisado.

BUENAS PRÁCTICAS DE CÓDIGO

Estilo y legibilidad: el mismo criterio en los dos

R

```r
# MAL
r<-datos|>filter(anio==2024)|>group_by(prov)
# BIEN
resultado <- datos |>
  filter(anio == 2024) |>
  group_by(prov) |>
  summarise(ingreso_medio =
    mean(ingreso, na.rm = TRUE))
```

Python

```python
# MAL
r=datos[datos.anio==2024].groupby("prov")
# BIEN
resultado = (
  datos
  .query("anio == 2024")
  .groupby("prov")
  .agg(ingreso_medio=("ingreso", "mean"))
)
```

Espacios alrededor de operadores y una operación por línea. Y no hace falta hacerlo a mano: styler lo arregla en R, ruff en Python.

BUENAS PRÁCTICAS DE CÓDIGO

Comentarios: explicar el por qué, no el qué

R

```r
# MAL: repite lo que dice el código
# Filtro por estado igual a 1
ocupados <- eph |> filter(ESTADO == 1)
# BIEN: explica el criterio
# Ocupados: ESTADO == 1 (manual EPH)
ocupados <- eph |> filter(ESTADO == 1)
# Limpieza de ingresos ----
```

Python

```python
# MAL: repite lo que dice el código
# Filtro por estado igual a 1
ocupados = eph[eph["ESTADO"] == 1]
# BIEN: explica el criterio
# Ocupados: ESTADO == 1 (manual EPH)
ocupados = eph[eph["ESTADO"] == 1]
# %% Limpieza de ingresos
```

Secciones: en R, un comentario que termina en cuatro guiones (----) aparece en el índice de RStudio. En Python, # %% arma una celda en VS Code.

BUENAS PRÁCTICAS DE CÓDIGO

Errores que rompen la reproducibilidad

Error	Cómo aparece en R	Cómo aparece en Python
Rutas absolutas	"C:/Users/..." en read.csv()	"C:/Users/..." en pd.read_csv()
Instalar dentro del script	install.packages() en el script	!pip install en el notebook
Depender de lo que quedó en memoria	cargar un .RData viejo	correr las celdas en desorden
No declarar los paquetes	un library() perdido en la línea 200	un import en el medio del código

La solución es la misma en los dos: rutas relativas, instalar en la consola, paquetes arriba de todo y correr siempre desde cero.

BUENAS PRÁCTICAS DE CÓDIGO

Arrancar siempre desde cero

R · RStudio

Que no guarde la sesión

Tools › Global Options › General: desmarcar «Restore .RData into workspace at startup» y poner «Save workspace to .RData on exit» en Never.

Reiniciar antes de entregar

Session › Restart R (Ctrl+Shift+F10) y correr el run_all.R.

Python · Jupyter / VS Code

Reiniciar y correr todo

En JupyterLab: Kernel › Restart Kernel and Run All Cells. En VS Code: Restart y Run All.

Mejor todavía

Pasar lo que funciona del notebook a scripts .py y correr el run_all.py.

Si no corre desde cero, no corre.

05

Entornos, dependencias y claves

Que el proyecto ande en otra computadora y que tus claves no se filtren.

ENTORNOS Y DEPENDENCIAS

«En mi compu anda»: declarar y congelar

R

```r
# 1. declarar: al inicio de cada script
library(tidyverse)
library(lubridate)
# 2. congelar: en la consola, una vez
renv::init() # entorno del proyecto
renv::snapshot() # escribe renv.lock
# 3. reproducir: en otra computadora
renv::restore()
```

Python

```python
# 1. declarar: al inicio de cada script
import pandas as pd
# 2. congelar: en la terminal, una vez
$ uv init # crea pyproject.toml
$ uv add pandas # agrega y anota versión
# 3. reproducir: en otra computadora
$ uv sync # instala desde uv.lock
```

Declarar es decir qué usás; congelar es anotar la versión exacta. install.packages() y pip install van en la consola, nunca en el script.

ENTORNOS Y DEPENDENCIAS

Claves y contraseñas: nunca dentro del código

R · .Renviron

```
# archivo .Renviron (carpeta del proyecto)
FRED_API_KEY=tu_clave
# en el script
clave <- Sys.getenv("FRED_API_KEY")
```

Python · .env

```
# archivo .env (carpeta del proyecto)
FRED_API_KEY=tu_clave
# en el script
import os
from dotenv import load_dotenv
load_dotenv()
clave = os.getenv("FRED_API_KEY")
```

Y .Renviron y .env van en el .gitignore. Una clave que llegó a GitHub hay que darla de baja: aunque la borres, queda en el historial.

ENTORNOS Y DEPENDENCIAS

Cómo crear el .Renviron, paso a paso

1. Crearlo

En la consola: usethis::edit_r_environ(scope = "project"). Crea el archivo .Renviron en la carpeta del proyecto y lo abre.

2. Escribir la clave

Una línea por clave, NOMBRE=valor, sin comillas ni espacios. Guardar.

3. Reiniciar R

Session › Restart R. R lee el .Renviron al arrancar (o readRenviron(".Renviron") sin reiniciar).

4. Que no se suba

usethis::use_git_ignore(".Renviron") lo agrega al .gitignore.

Así se ve el .Renviron

```
# .Renviron (en la carpeta del proyecto)
FRED_API_KEY=a1b2c3d4e5f6
OTRA_CLAVE=xyz789
```

Así se usa en el script

```
clave <- Sys.getenv("FRED_API_KEY")
# en Python: os.getenv("FRED_API_KEY")
```

06

APIs

Pedirle datos a un servidor desde el código.

APIS

¿Qué es una API?

Application Programming Interface: una forma de pedirle datos a un servidor con una dirección (URL) armada. En vez de bajar un CSV a mano, el código hace el pedido y recibe los datos.

A favor

Automatizable
Se actualiza sin intervención.

Reproducible
El pedido queda escrito en el código.

Actualizado
Siempre el dato más reciente.

Eficiente
Pedís solo lo que necesitás.

En contra

Internet
Sin conexión, no hay datos.

Cambia
Si la API cambia o se cae, tu script se rompe.

Claves
Algunas piden registro y clave.

Límites
Topes de registros por pedido.

APIS

Anatomía de un pedido

`https://api.bcra.gob.ar/estadisticas/v4.0/monetarias/1?desde=2026-09-01&hasta=2026-09-10`

1. URL base

El servidor: api.bcra.gob.ar

2. Endpoint

Qué recurso y qué versión: la variable 1, en v4.0

3. Parámetros

Filtros después del ?, unidos con &

4. Método

Casi siempre GET: «traeme datos»

La respuesta llega en JSON, «el CSV de las APIs»: pares de nombre y valor.

```
{"results": [{"idVariable": 1, "detalle": [
  {"fecha": "2026-09-10", "valor": 50506},
  {"fecha": "2026-09-09", "valor": 50617}, ...]}]}
```

APIS

Las APIs cambian: el BCRA va por la v4

v1.0	v2.0	v3.0	v4.0
dada de baja	**dada de baja**	**dada de baja**	**vigente**
15/06/2024	01/06/2025	28/02/2026	hoy

Lo que responde hoy la dirección de la versión 2 (probado el 06/10/2026):

```
$ GET api.bcra.gob.ar/estadisticas/v2.0/DatosVariable/1/...
HTTP 410
{"errorMessages": ["Método correspondiente a la v2 ha sido deprecado."]}
```

Si tu análisis dependía de esa dirección, hoy no corre. Por eso: versión explícita y copia local.

APIS · EJEMPLO 1

BCRA: el dólar en un rango de fechas

R

```r
library(tidyverse)
library(jsonlite)
url_bcra <- paste0(
  "https://api.bcra.gob.ar/",
  "estadisticascambiarias/v1.0/Cotizaciones/USD",
  "?fechaDesde=2026-08-01&fechaHasta=2026-09-30")
datos_bcra <- fromJSON(url_bcra)
tc_usd <- datos_bcra$results |>
  unnest(detalle) |>
  mutate(fecha = as.Date(fecha))
```

Python

```python
import pandas as pd
import requests
url_bcra = (
  "https://api.bcra.gob.ar/"
  "estadisticascambiarias/v1.0/Cotizaciones/USD"
  "?fechaDesde=2026-08-01&fechaHasta=2026-09-30")
respuesta = requests.get(url_bcra, timeout=30)
datos_bcra = respuesta.json()
tc_usd = pd.json_normalize(datos_bcra["results"],
  record_path="detalle", meta="fecha")
tc_usd["fecha"] = pd.to_datetime(tc_usd["fecha"])
```

Los dos devuelven la misma tabla: 42 días hábiles con fecha, moneda y cotización. unnest() y json_normalize() «aplanan» el JSON anidado.

APIS · EJEMPLO 1

Qué más se le puede pedir a la API del BCRA

Endpoint	Qué devuelve
/estadisticascambiarias/v1.0/Maestros/Divisas	El listado de monedas, con su código ISO
/estadisticascambiarias/v1.0/Cotizaciones?fecha=AAAA-MM-DD	Todas las cotizaciones de un día
/estadisticascambiarias/v1.0/Cotizaciones/USD?fechaDesde=…	La serie de una moneda (con fechaHasta)
/estadisticas/v4.0/monetarias	El listado de variables monetarias (1.610)
/estadisticas/v4.0/monetarias/{id}?desde=…&hasta=…	La serie de una variable

No pide clave

Se puede usar sin registrarse.

Paginación

limit y offset: cuántos registros traer y cuántos saltear. En cambiarias, limit va de 10 a 1.000.

APIS

Paginación: llegan 1.000 de 1.610

R

```r
url <- paste0("https://api.bcra.gob.ar/",
  "estadisticas/v4.0/monetarias")
partes <- list()
offset <- 0
repeat {
  r <- fromJSON(paste0(url,
    "?limit=1000&offset=", offset))
  partes[[length(partes) + 1]] <- r$results
  offset <- offset + 1000
  if (offset >= r$metadata$resultset$count) break
}
variables <- do.call(rbind, partes) # 1610
```

Python

```python
url = ("https://api.bcra.gob.ar/"
  "estadisticas/v4.0/monetarias")
partes, offset = [], 0
while True:
  r = requests.get(url, timeout=30, params={
    "limit": 1000, "offset": offset}).json()
  partes.append(pd.DataFrame(r["results"]))
  offset += 1000
  if offset >= r["metadata"]["resultset"]["count"]:
    break
variables = pd.concat(partes) # 1610
```

La respuesta trae cuántos registros hay en total (count). Se pide de a 1.000 hasta completarlos.

APIS · EJEMPLO 2

Banco Mundial: WDI en R, wbgapi en Python

R

```r
library(WDI)
# PBI per cápita (US$ constantes 2015)
pbi_arg <- WDI(
  country = "AR",
  indicator = c(pbi_pc = "NY.GDP.PCAP.KD"),
  start = 2000, end = 2023)
```

Python

```python
import wbgapi as wb
# PBI per cápita (US$ constantes 2015)
pbi_arg = wb.data.DataFrame(
  "NY.GDP.PCAP.KD", "ARG",
  time=range(2000, 2024),
  numericTimeKeys=True)
```

Mismos datos (2000: 10.632; 2023: 12.993), distintas reglas: WDI pide el país en dos letras y devuelve formato largo; wbgapi pide tres y devuelve formato ancho. Y range() no incluye el último año.

APIS

Cuatro reglas para cualquier API

La versión, a la vista

Que la URL diga v4.0: cuando cambie, vas a saber qué tocar.

Siempre una copia local

Lo que bajás va a datos/crudos/ con la fecha en el nombre. Si la API se cae, tu análisis sigue.

Las claves, fuera del código

En .Renviron o .env, y esos archivos en el .gitignore.

Mirá cuánto llegó

Compará con el count de la respuesta y paginá si hace falta.

```
write_csv(tc_usd, paste0("datos/crudos/tc_", Sys.Date(), ".csv"))
tc_usd.to_csv(f"datos/crudos/tc_{pd.Timestamp.today().date()}.csv", index=False)
```

07

Fechas: lo esencial

La regla de oro: AAAA-MM-DD. Leer fechas sin errores, arreglar las de Excel con janitor y agrupar por período.

FECHAS

Para la computadora, una fecha es un número: el 28/04/2026 es...

R

Date: días desde el **1/1/1970**

20571

Python

pandas: desde el **1/1/1970**

20571

Excel

días desde el **30/12/1899**

46140

Por eso se puede

Sumar (fecha + 30), comparar (fecha1 > fecha2) y ordenar cronológicamente. Y por eso Excel, que cuenta distinto, rompe las fechas: lo vemos en la próxima.

La convención: AAAA-MM-DD

En programación siempre se usa año-mes-día, también en nombres de archivo. Cualquier otra forma genera problemas y confusiones.

FECHAS

Fechas que llegan de Excel: el paquete janitor

R · janitor

```r
library(janitor)
excel_numeric_to_date(c(44927, 45000, 45366))
# "2023-01-01" "2023-03-15" "2024-03-15"
# sin paquetes, indicando el día cero de Excel
as.Date(45366, origin = "1899-12-30")
```

Python · pandas

```python
numeros = pd.Series([44927, 45000, 45366])
pd.to_datetime(numeros, unit="D",
  origin="1899-12-30")
# 2023-01-01 2023-03-15 2024-03-15
```

Si un archivo pasó por Excel, la fecha puede llegar como número (45366 en vez de 15/03/2024). En R, janitor lo arregla en una línea.

FECHAS

Leer fechas desde texto

El texto dice	R (lubridate)	Python (pandas)
"2026-04-28"	ymd(x)	pd.to_datetime(x)
"28/04/2026"	dmy(x)	pd.to_datetime(x, format="%d/%m/%Y")
"04-28-2026"	mdy(x)	pd.to_datetime(x, format="%m-%d-%Y")
"28/04/2026 14:30:00"	dmy_hms(x)	format="%d/%m/%Y %H:%M:%S"

La ventaja de lubridate

No hay que memorizar códigos: el nombre de la función dice el orden (d-m-y).

Los códigos sirven para los dos

%d %m %Y %H %M %S son los mismos en R base: as.Date(x, format = "%d/%m/%Y").

FECHAS

Dos trampas, dos lenguajes

01/02/2026

¿1 de febrero o 2 de enero?

R · lubridate

```
dmy("01/02/2026") # 2026-02-01
mdy("01/02/2026") # 2026-01-02
```

Python · pandas

```
pd.to_datetime("01/02/2026",
  dayfirst=True) # 2026-02-01
```

31/01 + 1 mes

¿28 de febrero? ¿3 de marzo? ¿Error?

R · lubridate

```
ymd("2026-01-31") + months(1) # NA
ymd("2026-01-31") %m+% months(1) # 28/02
```

Python · pandas

```
pd.Timestamp("2026-01-31") +
  pd.DateOffset(months=1) # 2026-02-28
```

Ninguno está mal: son decisiones distintas. Hay que saber cuál toma cada lenguaje.

FECHAS · EJEMPLO INTEGRADOR

Tipo de cambio promedio mensual, con los datos del BCRA

R

```r
tc_mensual <- tc_usd |>
  mutate(mes = floor_date(fecha, "month")) |>
  group_by(mes) |>
  summarise(tc_promedio =
    mean(tipoCotizacion, na.rm = TRUE))
```

Python

```python
tc_mensual = (tc_usd
  .assign(mes=tc_usd["fecha"].dt.to_period("M"))
  .groupby("mes", as_index=False)
  .agg(tc_promedio=("tipoCotizacion", "mean")))
```

Resultado en los dos: agosto 2026, $1.499,9 por dólar; septiembre 2026, $1.513,8. Mismo dato, mismo resultado.

RESUMEN

Toda la clase, en los dos lenguajes

Tema	R	Python
Raíz del proyecto	.Rproj · here()	carpeta abierta · pathlib
Molde de proyecto	usethis · cookiecutter::bake()	ccds · cookiecutter
Script maestro	run_all.R	run_all.py
Estilo	styler	ruff
Dependencias	library() · renv.lock	import · uv.lock
Claves	.Renviron	.env
Pedir a una API	jsonlite	requests
Fechas	lubridate	pandas

Cambian los nombres de las herramientas. Las ideas son las mismas.

PARA EL TP

Checklist antes de entregar

- [ ] Un README que diga qué es y cómo se corre
- [ ] Un script maestro que corre todo en orden
- [ ] Paquetes declarados al inicio; nada de instalar en el script
- [ ] Fechas como fechas, en formato AAAA-MM-DD
- [ ] datos/crudos/ intacto; todo lo demás sale de scripts
- [ ] Rutas relativas: nada de C:/Users/...
- [ ] Claves fuera del código: .Renviron o .env
- [ ] Lo que bajás de una API, guardado en datos/crudos/

La prueba final: pasale la carpeta a alguien del grupo y que la corra sin preguntarte nada.

BIBLIOGRAFÍA Y RECURSOS

Para seguir

Lecturas

Wickham, Çetinkaya-Rundel y Grolemund (2023)

R for Data Science, 2.ª ed. Cap. 6 (Workflow: scripts and projects) y cap. 17 (Dates and times). r4ds.hadley.nz

Wilson et al. (2017)

Good Enough Practices in Scientific Computing. PLoS Computational Biology, 13(6).

McKinney (2022)

Python for Data Analysis, 3.ª ed. Cap. 11 (Time Series). wesmckinney.com/book

Cookiecutter Data Science

cookiecutter-data-science.drivendata.org (página «Opinions»)

Documentación

Repositorio de ejemplo del curso

github.com/nsidicarocep/ejemplo_datos_empleo

APIs y fechas

bcra.gob.ar/apis-banco-central · WDI · wbgapi · cheatsheet de lubridate · pandas (Time series)

Estilo y entornos

style.tidyverse.org · peps.python.org/pep-0008 · rstudio.github.io/renv · docs.astral.sh/uv